Python Full Course - Complete Guide

A comprehensive journey from beginner to advanced Python developer.

Part 1: Python Fundamentals

1.1 Getting Started

What is Python?

Python is a versatile, beginner-friendly programming language used for web development, data science, machine learning, automation, scripting, and much more.

Installing Python:

1. Download Python from python.org
2. Run the installer and check "Add Python to PATH"
3. Verify installation: `python --version` in terminal

Code Editors:

- VS Code (recommended - free)
- PyCharm (powerful IDE)
- Jupyter Notebook (great for data science)

Your First Program:

```
print("Hello, World!")
```

1.2 Variables and Data Types

What are Variables?

Variables are containers for storing data values. Think of them as labeled boxes that hold information.

```python
# Creating variables
name = "Aslam"          # str (string/text)
age = 25                # int (integer/whole number)
height = 5.9            # float (decimal number)
is_student = True       # bool (boolean/true or false)

# Python is dynamically typed - no need to declare types
```

Main Data Types:

Type	Description	Example
`str`	Text/string	`"Hello"`, `'Python'`
`int`	Whole numbers	`10`, `-5`, `1000`
`float`	Decimal numbers	`3.14`, `-2.5`
`bool`	True or False	`True`, `False`
`list`	Ordered collection	`[1, 2, 3]`
`tuple`	Immutable collection	`(1, 2, 3)`
`dict`	Key-value pairs	`{"name": "Aslam"}`
`set`	Unique values	`{1, 2, 3}`
`None`	Null/no value	`None`

Type Checking:

```python
name = "Aslam"
print(type(name))   # <class 'str'>

age = 25
print(type(age))    # <class 'int'>
```

Type Conversion:

```
# String to integer
num_str = "100"
num = int(num_str)      # 100

# Integer to string
age = 25
age_str = str(age)      # "25"

# String to float
price = float("99.99")  # 99.99

# Float to integer (truncates)
x = int(3.7)            # 3
```

1.3 Operators

Arithmetic Operators:

```
a = 10
b = 3

print(a + b)    # 13  (addition)
print(a - b)    # 7   (subtraction)
print(a * b)    # 30  (multiplication)
print(a / b)    # 3.333... (division)
print(a // b)   # 3   (floor division)
print(a % b)    # 1   (modulus/remainder)
print(a ** b)   # 1000 (exponentiation)
```

Comparison Operators:

```python
x = 5
y = 10

print(x == y)   # False (equal to)
print(x != y)   # True  (not equal to)
print(x > y)    # False (greater than)
print(x < y)    # True  (less than)
print(x >= y)   # False (greater than or equal)
print(x <= y)   # True  (less than or equal)
```

Logical Operators:

```python
a = True
b = False

print(a and b)  # False (both must be True)
print(a or b)   # True  (at least one True)
print(not a)    # False (inverts the value)
```

Assignment Operators:

```python
x = 10      # assign value
x += 5      # x = x + 5 (now x = 15)
x -= 3      # x = x - 3 (now x = 12)
x *= 2      # x = x * 2 (now x = 24)
x /= 4      # x = x / 4 (now x = 6.0)
x //= 2     # x = x // 2 (floor division)
x **= 2     # x = x ** 2 (exponentiation)
x %= 3      # x = x % 3 (modulus)
```

Membership Operators:

```python
fruits = ["apple", "banana", "cherry"]

print("apple" in fruits)      # True
print("mango" not in fruits)  # True
```

Identity Operators:

```
a = [1, 2, 3]
b = [1, 2, 3]
c = a

print(a is c)       # True (same object)
print(a is b)       # False (different objects)
print(a == b)       # True (same values)
print(a is not b)   # True
```

1.4 Strings

Creating Strings:

```
# Three ways to create strings
text1 = "Hello"
text2 = 'Python'
text3 = """This is a
multi-line
string"""
```

String Indexing:

```
text = "Python"
#       012345

print(text[0])      # 'P' (first character)
print(text[1])      # 'y' (second character)
print(text[-1])     # 'n' (last character)
print(text[-2])     # 'o' (second from last)
```

String Slicing:

```python
text = "Hello World"
#       01234567890

print(text[0:5])    # "Hello"
print(text[:5])     # "Hello" (from start)
print(text[6:])     # "World" (to end)
print(text[::2])    # "HloWrd" (every 2nd char)
print(text[::-1])   # "dlroW olleH" (reversed)
print(text[2:8:2])  # "loW" (slice with step)
```

Important String Methods:

```python
text = "Hello, Python!"

# Case methods
text.upper()            # "HELLO, PYTHON!"
text.lower()            # "hello, python!"
text.title()            # "Hello, Python!"
text.capitalize()       # "Hello, python!"
text.swapcase()         # "hELLO, pYTHON!"

# Search and replace
text.find("Python")     # 7 (index where found)
text.find("Java")       # -1 (not found)
text.replace("Python", "World")  # "Hello, World!"
text.count("l")         # 2 (occurrences)

# Strip and split
"  hello  ".strip()     # "hello" (remove whitespace)
"  hello  ".lstrip()    # "hello  " (left strip)
"  hello  ".rstrip()    # "  hello" (right strip)
text.split(", ")        # ["Hello", "Python!"]
"-".join(["a", "b", "c"])  # "a-b-c"

# Check methods
text.startswith("Hello")  # True
text.endswith("!")        # True
"123".isdigit()           # True
"abc".isalpha()           # True
"abc123".isalnum()        # True
"   ".isspace()           # True

# Padding
"42".zfill(5)           # "00042"
"hi".ljust(10, "-")     # "hi--------"
"hi".rjust(10, "-")     # "--------hi"
"hi".center(10, "-")    # "----hi----"
```

String Formatting:

```
name = "Aslam"
age = 25
price = 1234.5678

# f-strings (Python 3.6+) - RECOMMENDED
message = f"My name is {name}, age {age}"
print(f"Price: ${price:.2f}")      # "Price: $1234.57"
print(f"Price: ${price:>10.2f}")   # "Price: $   1234.57"
print(f"Binary: {42:b}")           # "Binary: 101010"
print(f"Hex: {255:x}")             # "Hex: ff"

# .format() method
message = "My name is {}, age {}".format(name, age)
message = "My name is {n}, age {a}".format(n=name, a=age)

# % operator (older style)
message = "My name is %s, age %d" % (name, age)
```

Escape Characters:

```
print("Hello\nWorld")     # Newline
print("Hello\tWorld")     # Tab
print("She said \"Hi\"")  # Double quote
print('It\'s mine')       # Single quote
print("Path: C:\\Users")  # Backslash
print(r"C:\Users\new")    # Raw string (no escaping)
```

1.5 Input and Output

Output (print):

```python
# Basic print
print("Hello, World!")

# Multiple values
name = "Aslam"
age = 25
print("Name:", name, "Age:", age)

# Custom separator
print("one", "two", "three", sep="-")  # one-two-three

# Custom ending
print("Hello", end=" ")
print("World")  # Hello World (same line)

# Formatted output
print(f"Name: {name}, Age: {age}")
```

Input:

```python
# Get user input (always returns string)
name = input("Enter your name: ")
print(f"Hello, {name}!")

# Numeric input (need to convert)
age = int(input("Enter your age: "))
height = float(input("Enter your height: "))
```

Part 2: Control Flow

2.1 Conditional Statements (if-else)

if Statement:

```python
age = 18

if age >= 18:
    print("You are an adult")
```

if-else:

```python
age = 15

if age >= 18:
    print("You can vote")
else:
    print("You cannot vote yet")
```

if-elif-else:

```python
marks = 75

if marks >= 90:
    grade = "A+"
elif marks >= 80:
    grade = "A"
elif marks >= 70:
    grade = "B"
elif marks >= 60:
    grade = "C"
elif marks >= 50:
    grade = "D"
else:
    grade = "F"

print(f"Your grade: {grade}")
```

Nested if:

```python
age = 25
has_license = True

if age >= 18:
    if has_license:
        print("You can drive")
    else:
        print("You need a license")
else:
    print("You are too young to drive")
```

Ternary Operator:

```python
age = 20
status = "adult" if age >= 18 else "minor"
print(status)

# Can be chained
score = 85
result = "A" if score >= 90 else "B" if score >= 80 else "C"
```

Multiple Conditions:

```python
age = 25
has_ticket = True

# Using 'and'
if age >= 18 and has_ticket:
    print("You can enter")

# Using 'or'
if age < 12 or age > 65:
    print("You get a discount")

# Using 'not'
is_banned = False
if not is_banned:
    print("Welcome!")
```

Match Statement (Python 3.10+):

```python
command = "start"

match command:
    case "start":
        print("Starting...")
    case "stop":
        print("Stopping...")
    case "restart":
        print("Restarting...")
    case _:
        print("Unknown command")
```

2.2 Loops

for Loop:

```python
# Using range
for i in range(5):
    print(i)  # 0, 1, 2, 3, 4

# With start and end
for i in range(1, 6):
    print(i)  # 1, 2, 3, 4, 5

# With step
for i in range(0, 10, 2):
    print(i)  # 0, 2, 4, 6, 8

# Reverse
for i in range(5, 0, -1):
    print(i)  # 5, 4, 3, 2, 1

# Iterating over a list
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)

# Iterating over a string
for char in "Python":
    print(char)

# Using enumerate (index + value)
for index, fruit in enumerate(fruits):
    print(f"{index}: {fruit}")

# Using enumerate with start index
for index, fruit in enumerate(fruits, start=1):
    print(f"{index}: {fruit}")

# Iterating over dictionary
person = {"name": "Aslam", "age": 25}
for key in person:
    print(key, person[key])

for key, value in person.items():
    print(f"{key}: {value}")
```

```python
# Using zip (parallel iteration)
names = ["Alice", "Bob", "Charlie"]
ages = [25, 30, 35]
for name, age in zip(names, ages):
    print(f"{name} is {age}")
```

while Loop:

```python
# Basic while
count = 0
while count < 5:
    print(count)
    count += 1

# With condition
password = ""
while password != "secret":
    password = input("Enter password: ")
print("Access granted!")

# Infinite loop with break
while True:
    user_input = input("Enter 'quit' to exit: ")
    if user_input == "quit":
        break
    print(f"You entered: {user_input}")
```

Loop Control:

```python
# break - exit the loop
for i in range(10):
    if i == 5:
        break
    print(i)  # 0, 1, 2, 3, 4

# continue - skip to next iteration
for i in range(5):
    if i == 2:
        continue
    print(i)  # 0, 1, 3, 4

# pass - do nothing (placeholder)
for i in range(5):
    if i == 2:
        pass  # TODO: handle this case
    print(i)

# else with loop (executes if no break)
for i in range(5):
    print(i)
else:
    print("Loop completed normally!")

# else not executed if break occurs
for i in range(5):
    if i == 3:
        break
    print(i)
else:
    print("This won't print")  # Skipped due to break
```

Nested Loops:

```python
# Multiplication table
for i in range(1, 6):
    for j in range(1, 11):
        print(f"{i} x {j} = {i*j}")
    print("---")

# Pattern printing
for i in range(5):
    for j in range(i + 1):
        print("*", end="")
    print()
# Output:
# *
# **
# ***
# ****
# *****
```

2.3 Comprehensions

List Comprehension:

```python
# Traditional way
squares = []
for x in range(10):
    squares.append(x ** 2)

# List comprehension (one line)
squares = [x ** 2 for x in range(10)]
# [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

# With condition (filter)
evens = [x for x in range(20) if x % 2 == 0]
# [0, 2, 4, 6, 8, 10, 12, 14, 16, 18]

# With if-else (transform)
result = ["even" if x % 2 == 0 else "odd" for x in range(5)]
# ['even', 'odd', 'even', 'odd', 'even']

# Nested comprehension
matrix = [[j for j in range(3)] for i in range(3)]
# [[0, 1, 2], [0, 1, 2], [0, 1, 2]]

# Flattening a matrix
matrix = [[1, 2], [3, 4], [5, 6]]
flat = [num for row in matrix for num in row]
# [1, 2, 3, 4, 5, 6]
```

Dictionary Comprehension:

```python
# Number and its square
squares_dict = {x: x**2 for x in range(5)}
# {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}

# From two lists
keys = ["a", "b", "c"]
values = [1, 2, 3]
my_dict = {k: v for k, v in zip(keys, values)}
# {'a': 1, 'b': 2, 'c': 3}

# With condition
even_squares = {x: x**2 for x in range(10) if x % 2 == 0}
# {0: 0, 2: 4, 4: 16, 6: 36, 8: 64}
```

Set Comprehension:

```python
# Unique lengths
words = ["hello", "world", "hi", "hey"]
lengths = {len(word) for word in words}
# {2, 3, 5}
```

Generator Expression:

```python
# Memory efficient (doesn't store all values)
squares_gen = (x**2 for x in range(1000000))

# Iterate when needed
for square in squares_gen:
    if square > 100:
        break
    print(square)
```

Part 3: Data Structures

3.1 Lists

Creating Lists:

```
# Empty list
empty = []
empty = list()

# With values
numbers = [1, 2, 3, 4, 5]
fruits = ["apple", "banana", "cherry"]
mixed = [1, "hello", 3.14, True, None]
nested = [[1, 2], [3, 4], [5, 6]]
```

Accessing Elements:

```
fruits = ["apple", "banana", "cherry", "date", "elderberry"]

fruits[0]       # "apple" (first)
fruits[-1]      # "elderberry" (last)
fruits[1:3]     # ["banana", "cherry"]
fruits[:3]      # ["apple", "banana", "cherry"]
fruits[2:]      # ["cherry", "date", "elderberry"]
fruits[::2]     # ["apple", "cherry", "elderberry"]
fruits[::-1]    # reversed list
```

Modifying Lists:

```python
fruits = ["apple", "banana", "cherry"]

# Adding elements
fruits.append("date")              # Add to end
fruits.insert(1, "blueberry")      # Insert at index
fruits.extend(["fig", "grape"])    # Add multiple

# Removing elements
fruits.remove("banana")            # Remove by value
fruits.pop()                       # Remove & return last
fruits.pop(0)                      # Remove & return at index
del fruits[1]                      # Delete at index
fruits.clear()                     # Remove all

# Modifying elements
fruits[0] = "apricot"              # Change value
fruits[1:3] = ["x", "y", "z"]      # Replace slice
```

List Methods:

```python
numbers = [3, 1, 4, 1, 5, 9, 2, 6]

len(numbers)            # 8 (length)
numbers.sort()          # Sort in place (ascending)
numbers.sort(reverse=True)  # Sort descending
sorted(numbers)         # Return sorted copy
numbers.reverse()       # Reverse in place
numbers.count(1)        # 2 (occurrences)
numbers.index(5)        # Index of first 5
min(numbers)            # 1 (minimum)
max(numbers)            # 9 (maximum)
sum(numbers)            # 31 (sum)

# Copying
new_list = numbers.copy()
new_list = numbers[:]
new_list = list(numbers)

# Checking membership
5 in numbers            # True
10 not in numbers       # True
```

List Operations:

```python
a = [1, 2, 3]
b = [4, 5, 6]

# Concatenation
c = a + b               # [1, 2, 3, 4, 5, 6]

# Repetition
d = a * 3               # [1, 2, 3, 1, 2, 3, 1, 2, 3]

# Comparison
[1, 2] == [1, 2]        # True
[1, 2] < [1, 3]         # True (lexicographic)
```

3.2 Tuples

Tuples are like lists but immutable (cannot be changed after creation).

```
# Creating tuples
coordinates = (10, 20, 30)
single = (5,)              # Single element needs comma
empty = ()
from_list = tuple([1, 2, 3])

# Accessing (same as lists)
coordinates[0]      # 10
coordinates[-1]     # 30
coordinates[1:3]    # (20, 30)

# Unpacking
x, y, z = coordinates
print(x)  # 10

# Extended unpacking
first, *rest = (1, 2, 3, 4, 5)
# first = 1, rest = [2, 3, 4, 5]

*start, last = (1, 2, 3, 4, 5)
# start = [1, 2, 3, 4], last = 5

# Tuple methods
coordinates.count(10)   # 1
coordinates.index(20)   # 1

# Tuple operations
a = (1, 2)
b = (3, 4)
c = a + b               # (1, 2, 3, 4)
d = a * 2               # (1, 2, 1, 2)
```

When to Use Tuples:

- Data that shouldn't change
- Dictionary keys (lists can't be keys)

- Function return values
- Slightly more memory efficient than lists

Named Tuples:

```
from collections import namedtuple

Point = namedtuple('Point', ['x', 'y'])
p = Point(10, 20)

print(p.x)      # 10
print(p.y)      # 20
print(p[0])     # 10 (still indexable)
```

3.3 Sets

Sets are collections of unique values with no order.

```
# Creating sets
numbers = {1, 2, 3, 4, 5}
empty = set()           # {} creates dict, not set!
from_list = set([1, 2, 2, 3, 3, 3])  # {1, 2, 3}

# Duplicates automatically removed
nums = {1, 2, 2, 3, 3, 3}
print(nums)  # {1, 2, 3}

# Remove duplicates from list
my_list = [1, 2, 2, 3, 3, 3]
unique = list(set(my_list))  # [1, 2, 3]
```

Set Operations:

```
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}

# Union (all elements)
a | b               # {1, 2, 3, 4, 5, 6}
a.union(b)

# Intersection (common elements)
a & b               # {3, 4}
a.intersection(b)

# Difference (in a but not b)
a - b               # {1, 2}
a.difference(b)

# Symmetric difference (not in both)
a ^ b               # {1, 2, 5, 6}
a.symmetric_difference(b)

# Subset and superset
{1, 2} <= {1, 2, 3}      # True (subset)
{1, 2, 3} >= {1, 2}      # True (superset)
{1, 2}.issubset({1, 2, 3})
{1, 2, 3}.issuperset({1, 2})
```

Set Methods:

```
s = {1, 2, 3}

s.add(4)            # Add element
s.remove(2)         # Remove (error if not found)
s.discard(10)       # Remove (no error if not found)
s.pop()             # Remove and return arbitrary element
s.clear()           # Remove all

3 in s              # True (membership check)
len(s)              # Length
```

Frozen Sets:

```
# Immutable set (can be dict key or set element)
fs = frozenset([1, 2, 3])
# fs.add(4)  # Error - immutable
```

3.4 Dictionaries

Dictionaries store key-value pairs.

```
# Creating dictionaries
student = {
    "name": "Aslam",
    "age": 25,
    "city": "Dhaka"
}

empty = {}
empty = dict()
from_tuples = dict([("a", 1), ("b", 2)])
from_keys = dict.fromkeys(["a", "b", "c"], 0)  # All values = 0
```

Accessing and Modifying:

```python
student = {"name": "Aslam", "age": 25}

# Accessing
student["name"]              # "Aslam"
student.get("name")          # "Aslam"
student.get("address", "N/A") # "N/A" (default if not found)

# Adding/Modifying
student["address"] = "Dhaka"  # Add new key
student["age"] = 26           # Modify existing
student.update({"phone": "123", "email": "a@b.com"})

# Removing
del student["age"]
student.pop("address")        # Remove and return value
student.pop("xyz", "default") # With default (no error)
student.popitem()             # Remove last inserted
student.clear()               # Remove all

# Set default
student.setdefault("country", "Bangladesh")
# Returns value if key exists, otherwise sets and returns default
```

Dictionary Methods:

```python
student = {"name": "Aslam", "age": 25, "city": "Dhaka"}

student.keys()      # dict_keys(['name', 'age', 'city'])
student.values()    # dict_values(['Aslam', 25, 'Dhaka'])
student.items()     # dict_items([('name', 'Aslam'), ...])

"name" in student   # True (check key exists)
len(student)        # 3
```

Iterating:

```python
student = {"name": "Aslam", "age": 25}

# Keys only
for key in student:
    print(key)

# Keys and values
for key, value in student.items():
    print(f"{key}: {value}")

# Values only
for value in student.values():
    print(value)
```

Nested Dictionaries:

```python
students = {
    "101": {"name": "Aslam", "age": 25},
    "102": {"name": "Karim", "age": 23}
}

students["101"]["name"]  # "Aslam"

# Adding to nested
students["103"] = {"name": "Jamil", "age": 24}
students["101"]["grade"] = "A"
```

Dictionary Merging (Python 3.9+):

```python
a = {"x": 1, "y": 2}
b = {"y": 3, "z": 4}

merged = a | b          # {'x': 1, 'y': 3, 'z': 4}
a |= b                  # Update a in place
```

Part 4: Functions

4.1 Defining Functions

Basic Function:

```
# Define a function
def greet():
    print("Hello!")

# Call the function
greet()
```

With Parameters:

```
def greet(name):
    print(f"Hello, {name}!")

greet("Aslam")  # Hello, Aslam!
```

With Return Value:

```python
def add(a, b):
    return a + b

result = add(5, 3)  # 8

# Multiple returns
def calculate(a, b):
    return a + b, a - b, a * b

total, diff, product = calculate(10, 5)

# Early return
def check_age(age):
    if age < 0:
        return "Invalid age"
    if age >= 18:
        return "Adult"
    return "Minor"
```

Docstrings:

```python
def greet(name):
    """
    Greet a person by name.

    Args:
        name: The person's name

    Returns:
        A greeting string
    """
    return f"Hello, {name}!"

# Access docstring
print(greet.__doc__)
help(greet)
```

4.2 Parameter Types

Default Parameters:

```
def greet(name, message="Good morning"):
    print(f"{message}, {name}!")

greet("Aslam")                    # Good morning, Aslam!
greet("Aslam", "Good night")      # Good night, Aslam!
```

Keyword Arguments:

```
def info(name, age, city):
    print(f"{name}, {age}, {city}")

# Can be in any order when using keywords
info(age=25, city="Dhaka", name="Aslam")
```

*args (Variable Positional Arguments):

```
def total(*numbers):
    return sum(numbers)

total(1, 2, 3)       # 6
total(1, 2, 3, 4, 5) # 15

def greet(*names):
    for name in names:
        print(f"Hello, {name}!")

greet("Alice", "Bob", "Charlie")
```

**kwargs (Variable Keyword Arguments):

```python
def print_info(**info):
    for key, value in info.items():
        print(f"{key}: {value}")

print_info(name="Aslam", age=25, city="Dhaka")
```

Combined:

```python
def func(a, b, *args, **kwargs):
    print(f"a = {a}")
    print(f"b = {b}")
    print(f"args = {args}")
    print(f"kwargs = {kwargs}")

func(1, 2, 3, 4, 5, x=10, y=20)
# a = 1
# b = 2
# args = (3, 4, 5)
# kwargs = {'x': 10, 'y': 20}
```

Keyword-Only Arguments:

```python
def greet(name, *, formal=False):
    # Arguments after * must be keyword-only
    if formal:
        print(f"Good day, {name}.")
    else:
        print(f"Hi, {name}!")

greet("Aslam", formal=True)
# greet("Aslam", True)  # Error!
```

Positional-Only Arguments (Python 3.8+):

```python
def greet(name, /, greeting="Hello"):
    # Arguments before / must be positional-only
    print(f"{greeting}, {name}!")

greet("Aslam")
# greet(name="Aslam")  # Error!
```

4.3 Lambda Functions

Anonymous, one-line functions.

```python
# Regular function
def square(x):
    return x ** 2

# Lambda equivalent
square = lambda x: x ** 2

print(square(5))  # 25

# Multiple parameters
add = lambda a, b: a + b
print(add(3, 5))  # 8

# With conditional
is_even = lambda x: x % 2 == 0
print(is_even(4))  # True
```

Lambda Use Cases:

```python
# Sorting with key
students = [("Aslam", 85), ("Karim", 90), ("Jamil", 78)]
students.sort(key=lambda x: x[1])  # Sort by score

# With map
numbers = [1, 2, 3, 4, 5]
squares = list(map(lambda x: x**2, numbers))
# [1, 4, 9, 16, 25]

# With filter
evens = list(filter(lambda x: x % 2 == 0, numbers))
# [2, 4]

# With reduce
from functools import reduce
product = reduce(lambda a, b: a * b, numbers)
# 120 (1*2*3*4*5)
```

4.4 Scope and Global Variables

Local vs Global:

```python
x = 10  # Global

def test():
    x = 5       # Local (different variable)
    print(x)    # 5

test()
print(x)        # 10 (global unchanged)
```

Using global:

```
total = 0

def add_to_total(value):
    global total    # Declare as global
    total += value

add_to_total(10)
print(total)  # 10
```

Nested Functions and nonlocal:

```
def outer():
    x = 10

    def inner():
        nonlocal x  # Access outer's x
        x += 5

    inner()
    print(x)  # 15

outer()
```

LEGB Rule:

- Local: Inside current function
- Enclosing: Inside enclosing functions
- Global: Module level
- Built-in: Python built-ins

4.5 Higher-Order Functions

Functions that take or return functions.

map():

```python
numbers = [1, 2, 3, 4, 5]

# Apply function to each element
squares = list(map(lambda x: x**2, numbers))
# [1, 4, 9, 16, 25]

# With multiple iterables
a = [1, 2, 3]
b = [4, 5, 6]
sums = list(map(lambda x, y: x + y, a, b))
# [5, 7, 9]
```

filter():

```python
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# Keep elements where function returns True
evens = list(filter(lambda x: x % 2 == 0, numbers))
# [2, 4, 6, 8, 10]
```

reduce():

```python
from functools import reduce

numbers = [1, 2, 3, 4, 5]

# Reduce to single value
total = reduce(lambda a, b: a + b, numbers)
# 15

product = reduce(lambda a, b: a * b, numbers)
# 120
```

sorted() with key:

```python
words = ["banana", "apple", "cherry", "date"]

# Sort by length
sorted(words, key=len)
# ['date', 'apple', 'banana', 'cherry']

# Sort by last character
sorted(words, key=lambda x: x[-1])
```

4.6 Closures and Decorators

Closures:

```python
def multiplier(n):
    def multiply(x):
        return x * n
    return multiply

double = multiplier(2)
triple = multiplier(3)

print(double(5))  # 10
print(triple(5))  # 15
```

Decorators:

```python
# Basic decorator
def uppercase(func):
    def wrapper():
        result = func()
        return result.upper()
    return wrapper

@uppercase
def greet():
    return "hello"

print(greet())  # HELLO

# Decorator with arguments
def repeat(times):
    def decorator(func):
        def wrapper(*args, **kwargs):
            for _ in range(times):
                result = func(*args, **kwargs)
            return result
        return wrapper
    return decorator

@repeat(3)
def say_hello(name):
    print(f"Hello, {name}!")

say_hello("Aslam")  # Prints 3 times
```

Timer Decorator:

```python
import time

def timer(func):
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        print(f"{func.__name__} took {end-start:.4f} seconds")
        return result
    return wrapper

@timer
def slow_function():
    time.sleep(1)
    return "Done"

slow_function()
```

Using functools.wraps:

```python
from functools import wraps

def my_decorator(func):
    @wraps(func)  # Preserves function metadata
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)
    return wrapper
```

Part 5: Modules and Packages

5.1 Built-in Modules

math Module:

```python
import math

math.sqrt(16)       # 4.0 (square root)
math.pow(2, 3)      # 8.0 (power)
math.pi             # 3.14159...
math.e              # 2.71828...
math.ceil(4.2)      # 5 (round up)
math.floor(4.8)     # 4 (round down)
math.trunc(4.8)     # 4 (truncate)
math.factorial(5)   # 120
math.gcd(12, 18)    # 6 (greatest common divisor)
math.log(100, 10)   # 2.0 (logarithm)
math.sin(math.pi/2) # 1.0
math.degrees(math.pi) # 180.0
math.radians(180)   # 3.14159...
```

random Module:

```python
import random

random.random()             # 0-1 float
random.randint(1, 10)       # 1-10 inclusive integer
random.randrange(1, 10, 2)  # 1, 3, 5, 7, or 9
random.choice([1, 2, 3])    # Pick one randomly
random.choices([1, 2, 3], k=2)  # Pick k with replacement
random.sample([1, 2, 3, 4], 2)  # Pick 2 without replacement
random.shuffle([1, 2, 3])   # Shuffle in place
random.uniform(1.0, 10.0)   # Random float in range

# Reproducible randomness
random.seed(42)
```

datetime Module:

```python
from datetime import datetime, date, time, timedelta

# Current date/time
now = datetime.now()
today = date.today()
current_time = datetime.now().time()

# Creating specific dates
d = date(2026, 2, 23)
t = time(12, 30, 45)
dt = datetime(2026, 2, 23, 12, 30, 45)

# Formatting
now.strftime("%Y-%m-%d")        # "2026-02-23"
now.strftime("%d/%m/%Y %H:%M")  # "23/02/2026 12:38"
now.strftime("%B %d, %Y")       # "February 23, 2026"

# Parsing
datetime.strptime("2026-02-23", "%Y-%m-%d")

# Arithmetic
tomorrow = today + timedelta(days=1)
last_week = today - timedelta(weeks=1)
diff = datetime(2026, 12, 31) - now

# Attributes
now.year, now.month, now.day
now.hour, now.minute, now.second
now.weekday()  # 0=Monday, 6=Sunday
```

os Module:

```python
import os

os.getcwd()                 # Current directory
os.chdir("/path")           # Change directory
os.listdir(".")             # List files
os.path.exists("file.txt")  # Check if exists
os.path.isfile("file.txt")  # Is it a file?
os.path.isdir("folder")     # Is it a directory?
os.mkdir("new_folder")      # Create directory
os.makedirs("a/b/c")        # Create nested directories
os.remove("file.txt")       # Delete file
os.rmdir("folder")          # Delete empty directory
os.rename("old", "new")     # Rename
os.path.join("a", "b", "c") # "a/b/c" (path joining)
os.path.basename("/a/b/c")  # "c"
os.path.dirname("/a/b/c")   # "/a/b"
os.path.splitext("file.txt") # ("file", ".txt")
os.environ["HOME"]          # Environment variables
```

sys Module:

```python
import sys

sys.version             # Python version
sys.platform            # OS platform
sys.argv                # Command line arguments
sys.path                # Module search paths
sys.exit(0)             # Exit program
sys.stdin, sys.stdout, sys.stderr  # Standard streams
```

json Module:

```python
import json

# Python to JSON string
data = {"name": "Aslam", "age": 25}
json_str = json.dumps(data, indent=2)

# JSON string to Python
data = json.loads('{"name": "Aslam"}')

# File operations
with open("data.json", "w") as f:
    json.dump(data, f, indent=2)

with open("data.json", "r") as f:
    data = json.load(f)
```

re Module (Regular Expressions):

```python
import re

text = "My email is test@example.com"

# Search
match = re.search(r'\w+@\w+\.\w+', text)
if match:
    print(match.group())  # test@example.com

# Find all
emails = re.findall(r'\w+@\w+\.\w+', text)

# Replace
new_text = re.sub(r'\d+', 'X', "abc123def456")
# "abcXdefX"

# Split
parts = re.split(r'\s+', "hello   world")
# ["hello", "world"]

# Match (from start)
if re.match(r'My', text):
    print("Starts with 'My'")
```

5.2 Creating Your Own Modules

mymodule.py:

```python
"""My custom module."""

PI = 3.14159

def greet(name):
    """Greet someone."""
    return f"Hello, {name}!"

def add(a, b):
    """Add two numbers."""
    return a + b

class Calculator:
    """Simple calculator class."""

    def multiply(self, a, b):
        return a * b
```

main.py:

```python
# Import entire module
import mymodule

print(mymodule.greet("Aslam"))
print(mymodule.PI)

# Import specific items
from mymodule import greet, PI, Calculator
print(greet("Aslam"))

# Import with alias
import mymodule as mm
print(mm.greet("Aslam"))

# Import all (not recommended)
from mymodule import *
```

Package Structure:

```
mypackage/
    __init__.py
    module1.py
    module2.py
    subpackage/
        __init__.py
        module3.py
```

init.py:

```
# Can be empty or contain initialization code
from .module1 import func1
from .module2 import func2

__all__ = ['func1', 'func2']  # Controls "from package import *"
```

5.3 Installing Packages with pip

```
# Install package
pip install requests
pip install pandas numpy matplotlib

# Install specific version
pip install requests==2.28.0
pip install "requests>=2.25,<3.0"

# Upgrade package
pip install --upgrade requests

# Uninstall
pip uninstall requests

# List installed
pip list
pip freeze

# Show package info
pip show requests

# Install from requirements.txt
pip install -r requirements.txt

# Create requirements.txt
pip freeze > requirements.txt
```

Virtual Environments:

```
# Create virtual environment
python -m venv myenv

# Activate (Windows)
myenv\Scripts\activate

# Activate (Mac/Linux)
source myenv/bin/activate

# Deactivate
deactivate
```

Part 6: File Handling

6.1 Reading and Writing Files

Writing to Files:

```
# Traditional way
file = open("test.txt", "w")
file.write("Hello, World!\n")
file.write("Second line")
file.close()

# With context manager (recommended)
with open("test.txt", "w") as file:
    file.write("Hello, World!\n")
    file.write("Second line")
# File automatically closed

# Write multiple lines
lines = ["Line 1\n", "Line 2\n", "Line 3\n"]
with open("test.txt", "w") as file:
    file.writelines(lines)
```

Reading Files:

```
# Read entire file
with open("test.txt", "r") as file:
    content = file.read()
    print(content)

# Read line by line
with open("test.txt", "r") as file:
    for line in file:
        print(line.strip())

# Read as list of lines
with open("test.txt", "r") as file:
    lines = file.readlines()

# Read specific amount
with open("test.txt", "r") as file:
    chunk = file.read(100)  # First 100 characters
    line = file.readline()   # One line
```

File Modes:

Mode	Description
`"r"`	Read (default)
`"w"`	Write (overwrites)
`"a"`	Append (add to end)
`"x"`	Create (error if exists)
`"r+"`	Read and write
`"w+"`	Write and read (overwrites)
`"a+"`	Append and read
`"b"`	Binary mode
`"t"`	Text mode (default)

```
# Binary file
with open("image.png", "rb") as file:
    data = file.read()

with open("copy.png", "wb") as file:
    file.write(data)
```

File Position:

```
with open("test.txt", "r") as file:
    print(file.tell())      # Current position
    file.read(5)            # Read 5 characters
    print(file.tell())      # Now at position 5
    file.seek(0)            # Go back to start
    print(file.tell())      # Back at 0
```

6.2 Working with JSON

```python
import json

# Dictionary to JSON file
data = {
    "name": "Aslam",
    "age": 25,
    "cities": ["Dhaka", "Chittagong"],
    "active": True
}

with open("data.json", "w") as file:
    json.dump(data, file, indent=2)

# JSON file to dictionary
with open("data.json", "r") as file:
    loaded_data = json.load(file)
    print(loaded_data["name"])

# String conversion
json_string = json.dumps(data, indent=2)
data = json.loads(json_string)
```

6.3 Working with CSV

```python
import csv

# Writing CSV
with open("students.csv", "w", newline="") as file:
    writer = csv.writer(file)
    writer.writerow(["Name", "Age", "City"])
    writer.writerow(["Aslam", 25, "Dhaka"])
    writer.writerow(["Karim", 23, "Chittagong"])

    # Write multiple rows
    writer.writerows([
        ["Jamil", 24, "Sylhet"],
        ["Rahim", 22, "Khulna"]
    ])

# Reading CSV
with open("students.csv", "r") as file:
    reader = csv.reader(file)
    for row in reader:
        print(row)

# As dictionary
with open("students.csv", "r") as file:
    reader = csv.DictReader(file)
    for row in reader:
        print(row["Name"], row["Age"])

# Write dictionary
with open("students.csv", "w", newline="") as file:
    fieldnames = ["Name", "Age", "City"]
    writer = csv.DictWriter(file, fieldnames=fieldnames)
    writer.writeheader()
    writer.writerow({"Name": "Aslam", "Age": 25, "City": "Dhaka"})
```

6.4 Working with Paths

```python
from pathlib import Path

# Create path object
p = Path("folder/file.txt")
home = Path.home()
cwd = Path.cwd()

# Path operations
p.name          # "file.txt"
p.stem          # "file"
p.suffix        # ".txt"
p.parent        # Path("folder")
p.parts         # ("folder", "file.txt")

# Check existence
p.exists()
p.is_file()
p.is_dir()

# Create directories
Path("new/nested/folder").mkdir(parents=True, exist_ok=True)

# List files
for file in Path(".").iterdir():
    print(file)

# Glob patterns
for py_file in Path(".").glob("*.py"):
    print(py_file)

for py_file in Path(".").rglob("*.py"):  # Recursive
    print(py_file)

# Read/Write
p.write_text("Hello, World!")
content = p.read_text()

p.write_bytes(b"binary data")
```

```
data = p.read_bytes()

# Join paths
new_path = Path("folder") / "subfolder" / "file.txt"
```

Part 7: Error Handling

7.1 try-except

```
try:
    num = int(input("Enter a number: "))
    result = 10 / num
    print(f"Result: {result}")
except ValueError:
    print("Please enter a valid number!")
except ZeroDivisionError:
    print("Cannot divide by zero!")
except Exception as e:
    print(f"Error: {e}")
```

try-except-else-finally:

```
try:
    file = open("test.txt", "r")
    content = file.read()
except FileNotFoundError:
    print("File not found!")
else:
    print(content)  # Runs if no exception
finally:
    print("Cleanup")  # Always runs
    if 'file' in locals():
        file.close()
```

Multiple Exceptions:

```
try:
    # Some code
    pass
except (ValueError, TypeError) as e:
    print(f"Value or Type error: {e}")
except Exception as e:
    print(f"Other error: {e}")
```

Getting Exception Info:

```
import traceback

try:
    1 / 0
except Exception as e:
    print(f"Error type: {type(e).__name__}")
    print(f"Error message: {e}")
    print(f"Traceback: {traceback.format_exc()}")
```

7.2 Raising Exceptions

```
def validate_age(age):
    if age < 0:
        raise ValueError("Age cannot be negative!")
    if age > 150:
        raise ValueError("Age is unrealistic!")
    return age

try:
    validate_age(-5)
except ValueError as e:
    print(e)
```

Re-raising Exceptions:

```
try:
    # Some code
    raise ValueError("Something went wrong")
except ValueError:
    print("Logging error...")
    raise  # Re-raise the same exception
```

7.3 Custom Exceptions

```
class InvalidAgeError(Exception):
    """Raised when age is invalid."""
    pass

class NegativeAgeError(InvalidAgeError):
    """Raised when age is negative."""
    def __init__(self, age):
        self.age = age
        super().__init__(f"Age cannot be negative: {age}")

def set_age(age):
    if age < 0:
        raise NegativeAgeError(age)
    return age

try:
    set_age(-5)
except NegativeAgeError as e:
    print(e)
    print(f"The invalid age was: {e.age}")
```

7.4 Context Managers

```python
# Using with statement
with open("file.txt", "r") as f:
    content = f.read()
# File automatically closed

# Multiple context managers
with open("in.txt") as f_in, open("out.txt", "w") as f_out:
    f_out.write(f_in.read())

# Custom context manager (class-based)
class FileManager:
    def __init__(self, filename, mode):
        self.filename = filename
        self.mode = mode

    def __enter__(self):
        self.file = open(self.filename, self.mode)
        return self.file

    def __exit__(self, exc_type, exc_val, exc_tb):
        self.file.close()
        return False  # Don't suppress exceptions

with FileManager("test.txt", "w") as f:
    f.write("Hello!")

# Custom context manager (decorator-based)
from contextlib import contextmanager

@contextmanager
def file_manager(filename, mode):
    f = open(filename, mode)
    try:
        yield f
    finally:
        f.close()
```

```python
with file_manager("test.txt", "w") as f:
    f.write("Hello!")
```

Part 8: Object-Oriented Programming (OOP)

8.1 Classes and Objects

```python
# Define a class
class Student:
    # Class variable (shared by all instances)
    school = "Python High School"

    # Constructor (initializer)
    def __init__(self, name, age):
        # Instance variables
        self.name = name
        self.age = age

    # Instance method
    def introduce(self):
        print(f"I am {self.name}, age {self.age}")

    def study(self, subject):
        print(f"{self.name} is studying {subject}")

# Create objects (instances)
student1 = Student("Aslam", 25)
student2 = Student("Karim", 23)

# Access attributes
print(student1.name)        # Aslam
print(Student.school)       # Python High School

# Call methods
student1.introduce()        # I am Aslam, age 25
student1.study("Python")    # Aslam is studying Python
```

The init Method:

```python
class Person:
    def __init__(self, name, age=18):
        self.name = name
        self.age = age
        self.id = id(self)  # Unique identifier

p = Person("Aslam")
p = Person("Karim", 25)
```

8.2 Inheritance

```python
# Parent class (base class)
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        print("Some sound...")

    def info(self):
        print(f"I am {self.name}")

# Child class (derived class)
class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name)  # Call parent constructor
        self.breed = breed

    def speak(self):  # Override parent method
        print(f"{self.name} says Woof!")

class Cat(Animal):
    def speak(self):
        print(f"{self.name} says Meow!")

# Usage
dog = Dog("Tommy", "German Shepherd")
cat = Cat("Whiskers")

dog.speak()     # Tommy says Woof!
cat.speak()     # Whiskers says Meow!
dog.info()      # I am Tommy (inherited)

# Check inheritance
print(isinstance(dog, Dog))     # True
print(isinstance(dog, Animal))  # True
print(issubclass(Dog, Animal))  # True
```

Multiple Inheritance:

```python
class A:
    def method_a(self):
        print("Method A")

class B:
    def method_b(self):
        print("Method B")

class C(A, B):  # Inherits from both A and B
    def method_c(self):
        print("Method C")

c = C()
c.method_a()  # From A
c.method_b()  # From B
c.method_c()  # Own method

# Method Resolution Order (MRO)
print(C.__mro__)
```

8.3 Encapsulation

```python
class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner          # Public
        self._type = "Savings"      # Protected (convention)
        self.__balance = balance    # Private (name mangling)

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
            print(f"Deposited ${amount}")

    def withdraw(self, amount):
        if amount <= self.__balance:
            self.__balance -= amount
            print(f"Withdrew ${amount}")
        else:
            print("Insufficient funds!")

    def get_balance(self):
        return self.__balance

    # Property decorator
    @property
    def balance(self):
        return self.__balance

    @balance.setter
    def balance(self, value):
        if value >= 0:
            self.__balance = value

# Usage
account = BankAccount("Aslam", 1000)
account.deposit(500)
account.withdraw(200)
print(account.balance)  # 1300 (using property)
```

```
# Private attribute access (not recommended)
print(account._BankAccount__balance)  # Name mangling
```

8.4 Polymorphism

```
class Shape:
    def area(self):
        pass

class Rectangle(Shape):
    def __init__(self, width, height):
        self.width = width
        self.height = height

    def area(self):
        return self.width * self.height

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return 3.14159 * self.radius ** 2

# Polymorphism in action
shapes = [Rectangle(4, 5), Circle(3), Rectangle(2, 3)]

for shape in shapes:
    print(f"Area: {shape.area()}")

# Duck typing
def get_area(shape):
    return shape.area()  # Works with any object that has area()
```

8.5 Special Methods (Magic Methods)

```python
class Vector:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    # String representation
    def __str__(self):
        return f"Vector({self.x}, {self.y})"

    def __repr__(self):
        return f"Vector({self.x}, {self.y})"

    # Arithmetic operations
    def __add__(self, other):
        return Vector(self.x + other.x, self.y + other.y)

    def __sub__(self, other):
        return Vector(self.x - other.x, self.y - other.y)

    def __mul__(self, scalar):
        return Vector(self.x * scalar, self.y * scalar)

    # Comparison
    def __eq__(self, other):
        return self.x == other.x and self.y == other.y

    def __lt__(self, other):
        return (self.x**2 + self.y**2) < (other.x**2 + other.y**2)

    # Length
    def __len__(self):
        return int((self.x**2 + self.y**2) ** 0.5)

    # Indexing
    def __getitem__(self, index):
        if index == 0:
            return self.x
        elif index == 1:
```

```
            return self.y
        raise IndexError("Index out of range")

# Usage
v1 = Vector(3, 4)
v2 = Vector(1, 2)

print(v1)           # Vector(3, 4)
print(v1 + v2)      # Vector(4, 6)
print(v1 * 2)       # Vector(6, 8)
print(v1 == v2)     # False
print(len(v1))      # 5
print(v1[0])        # 3
```

Common Magic Methods:

Method	Description
`__init__`	Constructor
`__str__`	String for users
`__repr__`	String for developers
`__len__`	Length
`__getitem__`	Indexing `obj[key]`
`__setitem__`	Assignment `obj[key] = value`
`__delitem__`	Deletion `del obj[key]`
`__iter__`	Iteration
`__next__`	Next item
`__call__`	Make callable `obj()`
`__add__`, `__sub__`, `__mul__`	Arithmetic
`__eq__`, `__lt__`, `__gt__`	Comparison
`__enter__`, `__exit__`	Context manager

8.6 Static and Class Methods

```python
class Calculator:
    # Class variable
    pi = 3.14159

    def __init__(self, value=0):
        self.value = value

    # Instance method (needs self)
    def add(self, x):
        self.value += x
        return self

    # Static method (no self, no cls)
    @staticmethod
    def multiply(a, b):
        return a * b

    # Class method (has cls)
    @classmethod
    def circle_area(cls, radius):
        return cls.pi * radius ** 2

    # Alternative constructor
    @classmethod
    def from_string(cls, value_str):
        return cls(int(value_str))

# Usage
print(Calculator.multiply(5, 3))      # 15 (no instance needed)
print(Calculator.circle_area(5))      # 78.53975

calc = Calculator.from_string("100")
print(calc.value)                     # 100
```

8.7 Abstract Classes

```python
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

    @abstractmethod
    def perimeter(self):
        pass

    def describe(self):  # Regular method
        print(f"Area: {self.area()}, Perimeter: {self.perimeter()}")

class Rectangle(Shape):
    def __init__(self, width, height):
        self.width = width
        self.height = height

    def area(self):
        return self.width * self.height

    def perimeter(self):
        return 2 * (self.width + self.height)

# shape = Shape()  # Error! Can't instantiate abstract class
rect = Rectangle(4, 5)
rect.describe()  # Area: 20, Perimeter: 18
```

8.8 Dataclasses (Python 3.7+)

```python
from dataclasses import dataclass, field

@dataclass
class Student:
    name: str
    age: int
    grade: str = "A"
    courses: list = field(default_factory=list)

    def is_adult(self):
        return self.age >= 18

# Auto-generates __init__, __repr__, __eq__
s1 = Student("Aslam", 25)
s2 = Student("Aslam", 25)

print(s1)           # Student(name='Aslam', age=25, grade='A', courses=[])
print(s1 == s2)     # True
print(s1.is_adult()) # True

# Frozen (immutable)
@dataclass(frozen=True)
class Point:
    x: int
    y: int
```

Part 9: Advanced Topics

9.1 Iterators

```
# Custom iterator
class Counter:
    def __init__(self, start, end):
        self.current = start
        self.end = end

    def __iter__(self):
        return self

    def __next__(self):
        if self.current >= self.end:
            raise StopIteration
        self.current += 1
        return self.current - 1

# Usage
for num in Counter(1, 5):
    print(num)  # 1, 2, 3, 4

# Built-in iteration functions
numbers = [1, 2, 3, 4, 5]
it = iter(numbers)
print(next(it))  # 1
print(next(it))  # 2
```

9.2 Generators

```python
# Generator function
def count_up_to(n):
    count = 1
    while count <= n:
        yield count
        count += 1

# Usage
for num in count_up_to(5):
    print(num)  # 1, 2, 3, 4, 5

# Generator is memory efficient
gen = count_up_to(1000000)
print(next(gen))  # 1
print(next(gen))  # 2

# Generator expression
squares = (x**2 for x in range(10))
print(list(squares))

# Infinite generator
def infinite_counter():
    n = 0
    while True:
        yield n
        n += 1

# Send values to generator
def accumulator():
    total = 0
    while True:
        value = yield total
        if value is not None:
            total += value

acc = accumulator()
next(acc)           # Initialize
```

```
print(acc.send(10)) # 10
print(acc.send(5))  # 15
```

9.3 Async Programming

```python
import asyncio

# Async function (coroutine)
async def fetch_data(name, delay):
    print(f"Fetching {name}...")
    await asyncio.sleep(delay)  # Non-blocking wait
    print(f"Got {name}!")
    return f"{name} data"

# Run single coroutine
async def main():
    result = await fetch_data("user", 2)
    print(result)

asyncio.run(main())

# Run multiple concurrently
async def main():
    # Run concurrently
    results = await asyncio.gather(
        fetch_data("user", 2),
        fetch_data("posts", 1),
        fetch_data("comments", 3)
    )
    print(results)

asyncio.run(main())

# Create tasks
async def main():
    task1 = asyncio.create_task(fetch_data("user", 2))
    task2 = asyncio.create_task(fetch_data("posts", 1))

    result1 = await task1
    result2 = await task2
    print(result1, result2)
```

```
asyncio.run(main())
```

9.4 Threading and Multiprocessing

Threading:

```python
import threading
import time

def task(name, delay):
    print(f"Task {name} starting")
    time.sleep(delay)
    print(f"Task {name} done")

# Create threads
t1 = threading.Thread(target=task, args=("A", 2))
t2 = threading.Thread(target=task, args=("B", 1))

# Start threads
t1.start()
t2.start()

# Wait for completion
t1.join()
t2.join()

print("All tasks done")

# Thread with class
class MyThread(threading.Thread):
    def __init__(self, name):
        super().__init__()
        self.name = name

    def run(self):
        print(f"Thread {self.name} running")
```

Multiprocessing:

```python
from multiprocessing import Process, Pool
import os

def task(name):
    print(f"Process {name}, PID: {os.getpid()}")

# Single process
p = Process(target=task, args=("A",))
p.start()
p.join()

# Process pool
def square(n):
    return n ** 2

with Pool(4) as pool:
    results = pool.map(square, range(10))
    print(results)  # [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

9.5 Type Hints

```python
# Basic type hints
def greet(name: str) -> str:
    return f"Hello, {name}!"

def add(a: int, b: int) -> int:
    return a + b

# Complex types
from typing import List, Dict, Tuple, Optional, Union, Callable

def process_items(items: List[int]) -> Dict[str, int]:
    return {"sum": sum(items), "count": len(items)}

def get_user(user_id: int) -> Optional[str]:
    # Returns str or None
    pass

def parse(value: Union[str, int]) -> str:
    # Accepts str or int
    return str(value)

# Type aliases
Vector = List[float]
def scale(v: Vector, factor: float) -> Vector:
    return [x * factor for x in v]

# Callable
def apply(func: Callable[[int], int], value: int) -> int:
    return func(value)

# Generics
from typing import TypeVar, Generic

T = TypeVar('T')

class Stack(Generic[T]):
    def __init__(self):
        self.items: List[T] = []
```

```python
    def push(self, item: T) -> None:
        self.items.append(item)

    def pop(self) -> T:
        return self.items.pop()
```

Part 10: Common Libraries

10.1 requests (HTTP)

```python
import requests

# GET request
response = requests.get("https://api.github.com")
print(response.status_code)  # 200
print(response.json())

# With parameters
params = {"q": "python", "page": 1}
response = requests.get("https://api.github.com/search/repos", params=params)

# POST request
data = {"name": "Aslam", "age": 25}
response = requests.post("https://httpbin.org/post", json=data)

# Headers
headers = {"Authorization": "Bearer token123"}
response = requests.get("https://api.example.com", headers=headers)

# Error handling
try:
    response = requests.get("https://api.example.com", timeout=5)
    response.raise_for_status()  # Raise for 4xx/5xx
except requests.exceptions.RequestException as e:
    print(f"Error: {e}")
```

10.2 pandas (Data Analysis)

```python
import pandas as pd

# Create DataFrame
df = pd.DataFrame({
    "name": ["Aslam", "Karim", "Jamil"],
    "age": [25, 23, 24],
    "city": ["Dhaka", "Chittagong", "Sylhet"]
})

# Read from file
df = pd.read_csv("data.csv")
df = pd.read_excel("data.xlsx")
df = pd.read_json("data.json")

# Basic operations
df.head()           # First 5 rows
df.tail()           # Last 5 rows
df.info()           # Column info
df.describe()       # Statistics
df.shape            # (rows, columns)
df.columns          # Column names

# Selection
df["name"]          # Single column
df[["name", "age"]] # Multiple columns
df.loc[0]           # Row by label
df.iloc[0]          # Row by index
df.loc[0, "name"]   # Specific cell

# Filtering
df[df["age"] > 23]
df[(df["age"] > 23) & (df["city"] == "Dhaka")]
df.query("age > 23")

# Sorting
df.sort_values("age")
df.sort_values("age", ascending=False)
```

```python
# Grouping
df.groupby("city")["age"].mean()
df.groupby("city").agg({"age": ["mean", "max"]})

# Save
df.to_csv("output.csv", index=False)
df.to_excel("output.xlsx", index=False)
```

10.3 matplotlib (Visualization)

```
import matplotlib.pyplot as plt
import numpy as np

# Line plot
x = np.linspace(0, 10, 100)
y = np.sin(x)

plt.figure(figsize=(10, 6))
plt.plot(x, y, label="sin(x)", color="blue", linewidth=2)
plt.title("Sine Wave")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.grid(True)
plt.savefig("plot.png")
plt.show()

# Bar chart
categories = ["A", "B", "C", "D"]
values = [23, 45, 56, 78]

plt.bar(categories, values, color="skyblue")
plt.title("Bar Chart")
plt.show()

# Pie chart
plt.pie(values, labels=categories, autopct="%1.1f%%")
plt.title("Pie Chart")
plt.show()

# Subplots
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
axes[0].plot(x, np.sin(x))
axes[0].set_title("Sine")
axes[1].plot(x, np.cos(x))
axes[1].set_title("Cosine")
```

```
plt.tight_layout()
plt.show()
```

Part 11: Best Practices

11.1 Code Style (PEP 8)

```
# Naming conventions
variable_name = "snake_case for variables"
CONSTANT_NAME = "UPPER_CASE for constants"
function_name()  # snake_case for functions
ClassName        # PascalCase for classes
_private_var     # Leading underscore for private
__mangled_var    # Double underscore for name mangling

# Indentation: 4 spaces
if condition:
    do_something()

# Line length: max 79 characters
# Use line continuation for long lines
result = (first_value
          + second_value
          - third_value)

# Imports at top, grouped
import os
import sys

import numpy as np
import pandas as pd

from mymodule import my_function

# Whitespace
x = 1                     # Around assignment
func(a, b, c)            # After commas
if x == 1:               # Around operators
```

11.2 Documentation

```python
def calculate_area(length: float, width: float) -> float:
    """
    Calculate the area of a rectangle.

    Args:
        length: The length of the rectangle.
        width: The width of the rectangle.

    Returns:
        The area of the rectangle.

    Raises:
        ValueError: If length or width is negative.

    Examples:
        >>> calculate_area(5, 3)
        15.0
    """
    if length < 0 or width < 0:
        raise ValueError("Dimensions must be non-negative")
    return length * width
```

11.3 Testing

```python
# Using unittest
import unittest

def add(a, b):
    return a + b

class TestAdd(unittest.TestCase):
    def test_positive_numbers(self):
        self.assertEqual(add(2, 3), 5)

    def test_negative_numbers(self):
        self.assertEqual(add(-1, -1), -2)

    def test_zero(self):
        self.assertEqual(add(0, 0), 0)

if __name__ == "__main__":
    unittest.main()

# Using pytest (simpler)
def test_add_positive():
    assert add(2, 3) == 5

def test_add_negative():
    assert add(-1, -1) == -2
```

Part 12: Project Ideas

Beginner Level

1. Calculator - Basic arithmetic operations
2. Number Guessing Game - Random number guessing
3. To-Do List - Task management (console or file-based)

4. Password Generator - Generate strong passwords
5. Rock Paper Scissors - Classic game

Intermediate Level

6. Quiz Application - Multiple choice quiz
7. Weather App - Fetch weather data from API
8. URL Shortener - Shorten URLs
9. Web Scraper - Extract data from websites
10. Expense Tracker - Track income and expenses

Advanced Level

11. Blog Website - Using Django or Flask
12. Chatbot - AI-powered conversation
13. Image Processing - Photo editing with PIL/OpenCV
14. Data Analysis Dashboard - Interactive visualizations
15. REST API - Build your own API

Learning Resources

Official Documentation

- Python.org
- PEP 8 Style Guide

Online Courses

- Real Python
- Codecademy

- freeCodeCamp
- Coursera

Practice Platforms

- LeetCode
- HackerRank
- Codewars
- Project Euler

Books

- "Automate the Boring Stuff with Python" - Al Sweigart
- "Python Crash Course" - Eric Matthes
- "Fluent Python" - Luciano Ramalho
- "Effective Python" - Brett Slatkin

Tips for Success

1. **Practice daily** - Even 30 minutes helps
2. **Build projects** - Apply what you learn
3. **Read error messages** - They tell you what's wrong
4. **Use Google & Stack Overflow** - Everyone does
5. **Join communities** - Discord, Reddit, local meetups
6. **Read other people's code** - GitHub is full of examples
7. **Don't memorize, understand** - Concepts over syntax
8. **Start simple, add complexity** - Don't overwhelm yourself

---

> *Remember: Programming is a skill learned through practice. Be patient, stay curious, and enjoy the journey!* 🚀